

Evidencia GA4-220501095-AA2-EV01. Taller de conceptos y principios de programación orientada a objetos.

Instructor: Libardo Gómez Díaz

Aprendiz: Juan Camilo Herrera Castellanos

Ficha: 3336085

SENA

Servicio Aprendizaje Nacional

2026



Índice

1. introducción
2. Glosario de Terminología
3. Características y Pilares Básicos
4. Conclusiones
5. Referencias Bibliográficas

1. Introducción

El presente documento tiene como finalidad explorar y definir los fundamentos de la Programación Orientada a Objetos (POO), un paradigma de programación que permite desarrollar aplicaciones de manera organizada, reutilizable y cercana a la forma en que se representan los elementos del mundo real. Este enfoque facilita la creación de sistemas más eficientes mediante la utilización de clases, objetos, atributos y métodos que interactúan entre sí para resolver problemas de distinta complejidad.

A través de este taller se realizará un glosario con los principales términos utilizados en la Programación Orientada a Objetos, explicados con palabras propias para facilitar su comprensión. Además, se analizarán los conceptos, características y pilares fundamentales de este paradigma, entre ellos la abstracción, el encapsulamiento, la herencia y el polimorfismo, los cuales constituyen la base para el diseño y desarrollo de software moderno.

El estudio de estos conceptos permite comprender cómo se estructuran los programas orientados a objetos y cómo este paradigma contribuye a la construcción de soluciones informáticas más flexibles, mantenibles y escalables. De esta manera, se fortalece el conocimiento necesario para aplicar adecuadamente los principios de la programación orientada a objetos en futuros proyectos de desarrollo de software.

2. Glosario de Terminología

POO (Programación Orientada a Objetos)	Es una forma de organizar el código basándonos en "objetos" que contienen tanto datos como las acciones que pueden realizar, en lugar de solo listar instrucciones paso a paso.
Clase	Es la plantilla o plano de construcción. Define qué características y acciones tendrá un objeto, pero no es el objeto en sí mismo.
Objeto	Es la materialización o instancia de una clase. Es el elemento real con el que trabajamos en el programa.
Atributo	Son las características o datos que definen a un objeto (por ejemplo, el color o la marca de un carro).
Método	Es la acción o comportamiento que puede ejecutar un objeto (por ejemplo, "acelerar" o "frenar").
Herencia	Es la capacidad de crear nuevas clases basadas en otras existentes, permitiendo que la clase "hija" adquiera los atributos y métodos de la clase "padre" para no repetir código.
Encapsulamiento	Es la técnica de proteger los datos internos de un objeto, exponiendo solo lo necesario a través de una interfaz pública. Es como una "cápsula" que oculta los detalles de implementación.
Abstracción	Es el proceso de simplificar la realidad, enfocándose solo en las características esenciales de un objeto para el problema que estamos resolviendo e ignorando los detalles innecesarios.

Polimorfismo	Es la capacidad que tiene un objeto de tomar múltiples formas. En programación, permite que un mismo método ejecute acciones diferentes dependiendo de qué objeto lo llame.
Evento	Son sucesos que ocurren en el sistema, a menudo provocados por el usuario (como un clic en un botón) o por el sistema mismo, a los cuales el objeto debe responder.

3. Características y Pilares Básicos

Abstracción	Permite modelar el mundo real creando clases que reflejan los elementos relevantes de un problema.
Encapsulamiento	Garantiza la seguridad y organización, permitiendo que el objeto sea una unidad autónoma.
Herencia	Fomenta la reutilización del código. Si ya creé una clase "Vehículo", puedo crear "Moto" y "Carro" sin escribir todo de nuevo.
Polimorfismo	Ofrece flexibilidad al sistema, permitiendo que diferentes clases respondan a una misma instrucción de maneras distintas.

4. Conclusiones

La Programación Orientada a Objetos representa un cambio de enfoque fundamental en el desarrollo de software. Al permitir modelar entidades del mundo real mediante clases y objetos, facilita una transición lógica desde la identificación de problemas hasta su solución algorítmica, permitiendo que el código sea más organizado, comprensible y cercano a la realidad.

La aplicación de los pilares de la POO abstracción, encapsulamiento, herencia y polimorfismo es indispensable para gestionar la complejidad en sistemas de gran escala.

Estos principios no solo estructuran la información, sino que garantizan que el desarrollo sea modular, permitiendo que los componentes sean independientes y reutilizables.

El dominio de estos conceptos permite al desarrollador construir sistemas altamente eficientes, escalables y mantenibles en el tiempo. Al ocultar la complejidad interna mediante el encapsulamiento y permitir la especialización de comportamientos mediante la herencia y el polimorfismo, se reduce significativamente el riesgo de errores y se facilita el trabajo colaborativo en equipos de desarrollo.

En conclusión, la POO actúa como una metodología de diseño robusta que supera las limitaciones de los procedimientos tradicionales. Comprender estas bases es el requisito esencial para cualquier profesional en formación que busque desarrollar aplicaciones modernas, dinámicas y capaces de adaptarse a los requerimientos cambiantes de la industria tecnológica.

5. Referencias Bibliográficas

Aguilar, J. L. (2021). *Fundamentos de programación*. McGraw Hill Education.

Aguilar, L. J. (1998). *Programación orientada a objetos*. McGraw Hill Education.

Booch, G., Maksimchuk, R. A., Engle, M. W., Conallen, J., Young, B. J., & Houston, K. A. (2007). *Object-oriented Analysis and Design with Applications*. Addison-Wesley.

Grupo de Ejecución de la Formación Virtual. (s.f.). *Componente formativo: Conceptos básicos programación orientada a objetos*. SENA.

Rumbaugh, J., Jacobson, I., & Booch, G. (2004). *El lenguaje unificado de modelado*. Addison-Wesley.

Samuel. (2020). *¿Qué es la POO?* [Web log post]. Fundamentos de las POO.
<http://micanalsamuelc.blogspot.com/2017/03>.